{ }

ADVANCED JAVASCRIPT · COMPLETE COURSE

Advanced JavaScript
The Complete Handbook

8 days · 40 concepts · 80 real-world examples, all in one place

course.js

```
// Days 1-6: the original course
// Days 7-8: new advanced concepts
days = 8; concepts = 40; examples = 80;
```

CONTENTS

Everything in this handbook

Every concept follows the same shape: a definition, four key points, a compact example, and a second example drawn from real production code.

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 1: Functions & Scope

Definitions + two real-world examples for every concept

day1.js

```
// Run with: node day1.js
topics = ['functions', 'closures', 'IIFE', 'HOFs'];
```

Today's 5 concepts

01 Function Declarations vs Expressions vs Arrow Functions

02 Hoisting

03 Closures

04 IIFE — Immediately Invoked Function Expressions

05 Higher-Order Functions

Each concept: a clear definition, then two real-world code examples.

Declarations vs Expressions vs Arrow Functions

DEFINITION

JavaScript offers three ways to define a function: named declarations, expressions assigned to a variable, and concise arrow functions. They behave the same when called, but differ in hoisting and how they handle 'this'.

KEY POINTS

- Declarations are hoisted — safe to call before they appear in the file
- Expressions & arrows are NOT hoisted — must be defined before use
- Arrow functions have no own 'this' — great for short callbacks
- Same job, different rules — pick based on hoisting needs

EXAMPLE 1

day1.js

```
function greetDeclaration(name) {
  return `Hello, ${name}!`;
}

const greetExpression = function(name) {
  return `Hi, ${name}!`;
};

const squareArrow = (n) => n * n;

greetDeclaration('Asha');  // Hello, Asha!
squareArrow(5);            // 25
```

REAL-WORLD EXAMPLE

Example 2 — Checkout Module Relying on Hoisting

In a real checkout module, helper functions are often grouped at the bottom of the file for readability. A function declaration can safely be called before it's defined — a plain arrow/const can't.

checkout.js

```
// helper defined lower in the file, but callable now
console.log(isEligibleForDiscount(120));  // true

const withTax = cart.map((item) => ({
  ...item, price: applyTax(item.price),
}));

// ↓ function declaration — hoisted, order-independent
function isEligibleForDiscount(total) {
  return total > 100;
}

// ↓ arrow assigned to const — usable only after this line
const applyTax = (price) => +(price * 1.18).toFixed(2);
```

Why it matters: *Teams often group utilities at the bottom of a file. Knowing which style survives that ordering prevents 'is not a function' bugs during refactors.*

Hoisting

DEFINITION

During compilation, JS moves declarations to the top of their scope. Function declarations are hoisted completely (name + body). var is hoisted and initialized to undefined. let/const are hoisted but stay uninitialized in the 'Temporal Dead Zone' until their line runs.

KEY POINTS

- Function declarations → fully usable before their line in the file
- var → hoisted, but value is undefined until assigned
- let/const → hoisted into the Temporal Dead Zone (TDZ) — accessing early throws
- Arrow/expression functions follow their variable's hoisting rule (var/let/const)

EXAMPLE 1

day1.js

```
console.log(sayHi());  // Works!

function sayHi() {
  return "Hi there";
}

try {
  console.log(sayBye());  // ✗ error
} catch (err) {
  console.log(err.message);
  // Cannot access 'sayBye' before initialization
}
const sayBye = () => "Bye now";
```

REAL-WORLD EXAMPLE

Example 2 — A Config Load-Order Bug in Production

A very common real bug: someone reorders code during a refactor and a var silently becomes undefined instead of throwing, while a let/const correctly fails fast with a clear TDZ error.

config.js

```
// ✗ silent bug — var is hoisted as undefined, no error thrown
console.log('API base:', API_BASE);  // undefined
var API_BASE = 'https://api.shop.com';

// ✓ TDZ catches the same mistake immediately
try {
  console.log(userRole);
} catch (e) {
  console.log(e.message);
  // Cannot access 'userRole' before initialization
}
let userRole = 'admin';
```

Why it matters: *Prefer let/const over var: a loud TDZ error during development beats a silent 'undefined' that only surfaces as a bug in production.*

Closures

DEFINITION

A closure is a function bundled with references to its surrounding (lexical) scope. The inner function keeps access to the outer variables even after the outer function has already returned.

KEY POINTS

- Lets a function 'remember' variables from where it was created
- Perfect for private state — no global or public access needed
- Powers patterns like counters, bank accounts, and memoized caches
- Each call to the outer function creates a fresh, independent closure

EXAMPLE 1

day1.js

```
function createBankAccount(initialBalance) {
  let balance = initialBalance;
  return {
    deposit(amount) {
      balance += amount;
      return balance;
    },
    getBalance: () => balance,
  };
}

const account = createBankAccount(1000);
account.deposit(500);    // 1500
account.balance;    // undefined!
```

REAL-WORLD EXAMPLE

Example 2 — Click Handlers for a Rendered Product List

A storefront renders a list of product buttons in a loop and attaches a click handler to each. With var, every handler shares one variable; with let, each handler closes over its own index — the exact bug from Day 1's practice.

product-list.js

```
// ✗ every button alerts the SAME last index
for (var i = 0; i < buttons.length; i++) {
  buttons[i].onclick = () =>
    alert(`Product ${i}`);  // always the last i
}

// ✓ each handler closes over its own i
for (let i = 0; i < buttons.length; i++) {
  buttons[i].onclick = () =>
    alert(`Product ${i}`);  // correct index each time
}
```

Why it matters: *This exact bug shows up anywhere handlers are attached inside a loop — pagination buttons, table row actions, generated form fields.*

IIFE — Immediately Invoked Function Expression

DEFINITION

A function expression that is defined and executed immediately, in one step. It creates its own private scope, so any variables inside never leak into the surrounding code.

KEY POINTS

- Runs immediately — no separate call needed
- Creates an isolated scope — keeps helper variables private
- Classic way to avoid polluting the global namespace
- Precursor to today's ES Modules, which solve the same problem

EXAMPLE 1

day1.js

```
const calculator = (function () {
  let total = 0;  // private
  return {
    add(n) { total += n; return total; },
    reset() { total = 0; },
  };
})();

calculator.add(10);  // 10
console.log(total);  // ReferenceError — private!
```

REAL-WORLD EXAMPLE

Example 2 — A Self-Contained Analytics Snippet

Third-party embed scripts (analytics, chat widgets, ad trackers) are dropped into many different sites. An IIFE lets them set up their own state and expose one safe global, without clashing with the host page's variables.

tracker-widget.js

```
(function (window, document) {
  const trackerId = 'UA-10428';  // private
  let queue = [];

  function track(event) {
    queue.push(event);
  }

  // only ONE global is exposed to the host site
  window.myTracker = { track };
})(window, document);

myTracker.track('page_view');
```

Why it matters: *This is literally how classic embeddable scripts (old Google Analytics, chat widgets) avoid breaking the host page's own variables.*

Higher-Order Functions

DEFINITION

A function that takes another function as an argument, returns a function, or both. They let you treat behavior as data — passing logic around just like a value.

KEY POINTS

- Takes a function as a parameter (e.g. array.map(fn))
- Or returns a new function (e.g. a function factory)
- map / filter / reduce are the most common built-in examples
- Foundation for middleware, decorators, and functional pipelines

EXAMPLE 1

day1.js

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map((n) => n * 2);
const evens = numbers.filter((n) => n % 2 === 0);
const sum = numbers.reduce((t, n) => t + n, 0);

doubled;  // [2, 4, 6, 8, 10]
evens;    // [2, 4]
sum;      // 15
```

REAL-WORLD EXAMPLE

Example 2 — A Logging Wrapper for Any API Call

A higher-order function can wrap ANY function to add cross-cutting behavior — like timing or logging — without touching the original function's code. This is the same idea behind Express middleware.

with-logging.js

```
function withLogging(fn) {
  return function (...args) {
    console.time(fn.name);
    const result = fn(...args);
    console.timeEnd(fn.name);
    return result;
  };
}

function fetchUser(id) { /* ...api call... */ }

const fetchUserSafe = withLogging(fetchUser);
fetchUserSafe(42);  // same behavior, now timed
```

Why it matters: *This exact wrapping pattern powers Express middleware, React HOCs, and API client interceptors used in production apps every day.*

RECAP

Day 1 in one glance

- ✓ Functions: declarations hoist, expressions/arrows don't
- ✓ Hoisting: var → undefined silently, let/const → loud TDZ error
- ✓ Closures: inner functions remember outer variables — private state
- ✓ IIFE: run-and-isolate pattern, precursor to ES Modules
- ✓ Higher-order functions: pass or return functions as data

Practice: build an ID generator with closures, fix the var-loop bug, and write your own repeat(n, fn) higher-order function.

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 2: Objects & Prototypes

Definitions + two real-world examples for every concept

day2.js

```
// Run with: node day2.js
topics = ['creation patterns', 'prototypes', 'classes'];
```

Today's 5 concepts

01 Object Creation Patterns — literals, constructors, factories

02 Prototypal Inheritance & the Prototype Chain

03 Object.create, Object.assign & Object.freeze

04 Getters & Setters

05 ES6 Classes, Inheritance & Private Fields

Each concept: a clear definition, then two real-world code examples.

Object Creation Patterns

DEFINITION

JavaScript objects can be created three main ways: object literals for one-off objects, constructor functions with 'new' for many similar instances sharing methods, and factory functions that simply return a plain object.

KEY POINTS

- Object literal — fastest way to make a single, unique object
- Constructor + new — instances share methods via the prototype
- Factory function — returns a plain object, no 'new' or 'this' needed
- Constructors are more memory-efficient at scale (shared methods)

EXAMPLE 1

day2.js

```
// object literal
const user1 = { name: 'Asha', role: 'admin' };

// constructor function
function User(name, role) {
  this.name = name;
  this.role = role;
}
User.prototype.describe = function() {
  return `${this.name} (${this.role})`;
};
const user2 = new User('Ravi', 'editor');

// factory function
const createUser = (name, role) => ({ name, role });
```

REAL-WORLD EXAMPLE

Example 2 — Choosing a Pattern for Account Objects

In an auth system with thousands of user objects, a constructor keeps shared logic (like password checks) on one prototype instead of duplicating it per user, while a lightweight factory suits simple, role-based config objects.

accounts.js

```
// ✅ constructor — shared method lives once on the prototype
function Account(email, passwordHash) {
  this.email = email;
  this.passwordHash = passwordHash;
}
Account.prototype.checkPassword = function(hash) {
  return hash === this.passwordHash;
};
// 10,000 accounts share ONE checkPassword function in memory

// ✅ factory — quick, no 'new', good for simple role configs
const createGuestSession = () => ({
  role: 'guest', permissions: [], expiresIn: 3600,
});
```

Why it matters: *At scale, method placement matters: prototype methods are shared once, while methods defined inside every object literal or factory are duplicated per instance.*

Prototypal Inheritance & the Prototype Chain

DEFINITION

Every object has an internal link to another object — its prototype. When a property or method isn't found on the object itself, JavaScript walks up this chain until it finds it, or reaches null.

KEY POINTS

- Object.getPrototypeOf(obj) reveals the link (legacy: __proto__)
- Methods on Constructor.prototype are shared by every instance
- The chain ends at Object.prototype, then null
- This mechanism is what 'class' inheritance is built on top of

EXAMPLE 1

day2.js

```
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return `${this.name} makes a sound`;
};

function Dog(name) { Animal.call(this, name); }
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.bark = function() {
  return `${this.name} says Woof!`;
};

const rex = new Dog('Rex');
rex.speak();  // found up the chain
```

REAL-WORLD EXAMPLE

Example 2 — Shared Behavior Across UI Components

A vanilla-JS widget library defines common behavior once on a base Component prototype (logging, event binding) so that every specific widget — buttons, modals, tooltips — inherits it without copying code.

components.js

```
function Component(el) { this.el = el; }
Component.prototype.log = function(msg) {
  console.log(`[${this.constructor.name}] ${msg}`);
};

function IconButton(el, icon) {
  Component.call(this, el);
  this.icon = icon;
}
IconButton.prototype = Object.create(Component.prototype);

const saveBtn = new IconButton(btnEl, 'save');
saveBtn.log('clicked');  // inherited, not duplicated
```

Why it matters: *Every framework's component model — and the DOM's own HTMLElement hierarchy — relies on exactly this chain instead of copy-pasting methods per widget.*

Object.create, Object.assign & Object.freeze

DEFINITION

Three built-in utilities for controlling objects: Object.create(proto) builds a new object with a specific prototype, Object.assign(target, ...sources) shallow-copies properties into a target, and Object.freeze(obj) locks an object against further changes.

KEY POINTS

- Object.create(proto) — manual prototype wiring, no constructor needed
- Object.assign(target, ...src) — merge configs or shallow-clone objects
- Object.freeze(obj) — makes top-level properties read-only
- Frozen ≠ deeply immutable — nested objects can still be mutated

EXAMPLE 1

day2.js

```
// merging default options with user overrides
const defaults = { theme: 'light', retries: 3 };
const userOptions = { theme: 'dark' };

const config = Object.assign({}, defaults,
userOptions);
// { theme: 'dark', retries: 3 }

Object.freeze(config);
config.theme = 'blue';  // silently ignored
console.log(config.theme);  // still 'dark'
```

REAL-WORLD EXAMPLE

Example 2 — Immutable App State Updates

State-management libraries never mutate state directly. Every update returns a brand-new object via spread/Object.assign, and app-wide constants are frozen at startup so no module can accidentally change shared config.

store.js

```
// ✅ frozen constants — safe to import anywhere
const APP_CONFIG = Object.freeze({
  apiVersion: 'v2', maxUploadMB: 25,
});

// ✅ reducer never mutates — returns a new object each time
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return { ...state, items: [...state.items, action.item] };
    default:
      return state;
  }
}
```

Why it matters: *Redux, Zustand, and React state all lean on this 'never mutate, always return new' rule — it's what makes undo/redo and change-detection reliable.*

Getters & Setters

DEFINITION

Getters and setters are special methods that run when a property is read or written, while still looking like a plain property to whoever is using the object. They let you compute values on read, or validate/react on write.

KEY POINTS

- Defined with the get / set keywords in an object literal or class
- get — compute a value on the fly instead of storing it directly
- set — validate or transform a value before it's stored
- Caller uses normal dot syntax (obj.prop) — no extra method calls

EXAMPLE 1

day2.js

```
const temperature = {
  _celsius: 0,
  get celsius() { return this._celsius; },
  set celsius(value) { this._celsius = value; },
  get fahrenheit() {
    return this._celsius * 9/5 + 32;
  },
};

temperature.celsius = 25;
temperature.fahrenheit;  // 77 — computed on read
```

REAL-WORLD EXAMPLE

Example 2 — Validating Input in a Signup Form

A User class uses a setter to reject an invalid email the moment it's assigned — no separate validate() call to remember — and a getter to compute a display name on demand instead of storing a redundant field.

user.js

```
class User {
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }
  set email(value) {
    if (!value.includes('@')) {
      throw new Error('Invalid email');
    }
    this._email = value;
  }
  get displayName() {
    return `${this.firstName} ${this.lastName}`;
  }
}
```

Why it matters: *Catching bad data at the moment of assignment — not in a separate validation pass — is exactly how real signup and profile-edit forms stay consistent.*

ES6 Classes, Inheritance & Private Fields

DEFINITION

The class keyword is syntactic sugar over prototypal inheritance — a cleaner way to define a constructor and shared methods. extends/super enable single inheritance, and #privateFields give truly inaccessible internal state.

KEY POINTS

- constructor, methods, and static members in one clean block
- extends + super() to inherit and extend a parent class
- #privateField — real privacy, unlike a closure-based convention
- Under the hood, classes still build the same prototype chain

EXAMPLE 1

day2.js

```
class Shape {
  constructor(name) { this.name = name; }
  describe() { return `This is a ${this.name}`; }
}

class Circle extends Shape {
  #radius;  // truly private
  constructor(radius) {
    super('circle');
    this.#radius = radius;
  }
  get area() { return Math.PI * this.#radius ** 2; }
}
```

REAL-WORLD EXAMPLE

Example 2 — Polymorphic Shipping in an Order System

A base Product class defines a default calculateShipping(). DigitalProduct and PhysicalProduct override it with their own logic — the order system calls the same method name on every product without checking its type.

products.js

```
class Product {
  constructor(name, price) { this.name = name; this.price = price; }
  calculateShipping() { return 0; }
}

class PhysicalProduct extends Product {
  constructor(name, price, weightKg) {
    super(name, price);
    this.weightKg = weightKg;
  }
  calculateShipping() { return this.weightKg * 2.5; }
}

const cart = [new Product('E-book', 9), new PhysicalProduct('Mug', 12, 0.4)];
cart.reduce((sum, p) => sum + p.calculateShipping(), 0);
```

Why it matters: *This is polymorphism doing real work: checkout code stays simple because every product type answers 'calculateShipping()' in its own way.*

RECAP

Day 2 in one glance

- ✓ Creation patterns: literal, constructor, or factory — pick by scale
- ✓ Prototype chain: shared methods live once, found by lookup
- ✓ Object.create/assign/freeze: build, merge, and lock down objects
- ✓ Getters/setters: validate on write, compute on read, plain syntax
- ✓ Classes: extends/super for inheritance, #fields for true privacy

Practice: build a Stack class with private state, and a Shape hierarchy where each subclass computes its own area().

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 3: Asynchronous JavaScript

Definitions + two real-world examples for every concept

day3.js

```
// Run with: node day3.js
topics = ['event loop', 'promises', 'async/await'];
```

Today's 5 concepts

01 Callbacks & the Event Loop

02 Promises — then, catch & finally

03 async / await

04 Promise.all, race, allSettled & any

05 Generators & Iterators

Each concept: a clear definition, then two real-world code examples.

Callbacks & the Event Loop

DEFINITION

JS runs on one thread. The event loop lets it handle slow operations (timers, network, file I/O) without blocking: it queues a callback to run once the current call stack is empty. Microtasks (Promises) always drain before the next macrotask (setTimeout).

KEY POINTS

- Call stack finishes ALL synchronous code first, no exceptions
- Async work is handed off, then its callback is queued for later
- Microtask queue (Promises) empties completely before each macrotask
- Deeply nested callbacks ('callback hell') are what Promises fixed

EXAMPLE 1

day3.js

```
console.log('1: sync start');

setTimeout(() => console.log('4: timeout'), 0);

Promise.resolve().then(() =>
  console.log('3: microtask'));

console.log('2: sync end');

// output order:
// 1: sync start
// 2: sync end
// 3: microtask   ← Promise beats setTimeout
// 4: timeout
```

REAL-WORLD EXAMPLE

Example 2 — Escaping Callback Hell in a Real App

A dashboard needs a user, then their profile, then their settings — each depending on the last. Nested callbacks turn into a 'pyramid of doom' that's hard to read and even harder to add error handling to.

load-dashboard.js

```
// ✗ callback hell — nested, hard to follow, error-prone
getUser(id, (err, user) => {
  if (err) return handleError(err);
  getProfile(user.id, (err, profile) => {
    if (err) return handleError(err);
    getSettings(profile.id, (err, settings) => {
      if (err) return handleError(err);
      renderDashboard(user, profile, settings);
    });
  });
});
// one handleError copied 3 times — easy to miss a case
```

Why it matters: *This exact pyramid shape in a real codebase is why Promises (next slide) and async/await exist — same steps, flat and readable.*

Promises

DEFINITION

A Promise represents the eventual result of an async operation. It starts pending, then settles once — either fulfilled (success) or rejected (failure) — and lets you chain .then()/.catch()/.finally() instead of nesting callbacks.

KEY POINTS

- Three states: pending → fulfilled or rejected, and it never changes again
- .then() always returns a new promise — that's what enables chaining
- .catch() catches a rejection from anywhere earlier in the chain
- .finally() runs no matter what — perfect for cleanup logic

EXAMPLE 1

day3.js

```
function wait(ms) {
  return new Promise((resolve) => {
    setTimeout(() => resolve('done!'), ms);
  });
}

wait(500)
  .then((result) => {
    console.log(result);  // done!
    return result.toUpperCase();
  })
  .then((upper) => console.log(upper))
  .catch((err) => console.log('Error:', err))
  .finally(() => console.log('cleanup'));
```

REAL-WORLD EXAMPLE

Example 2 — Chaining Real API Calls

A storefront page fetches the logged-in user, then that user's orders, then computes a lifetime total — three dependent network calls chained with .then(), with a single .catch() covering any failure in the whole sequence.

order-summary.js

```
fetch('/api/me')
  .then((res) => res.json())
  .then((user) => fetch(`/api/orders?user=${user.id}`))
  .then((res) => res.json())
  .then((orders) => {
    const total = orders.reduce((sum, o) => sum + o.amount, 0);
    renderTotal(total);
  })
  .catch((err) => {
    // ONE place catches a failure from ANY step above
    showErrorBanner('Could not load orders');
  });
```

Why it matters: *This single-catch pattern is exactly how real frontend data-fetching chains handle network failures without repeating error-handling code at every step.*

async / await

DEFINITION

Syntax sugar over Promises: 'await' pauses an async function until a Promise settles, letting asynchronous code read top-to-bottom like synchronous code — while still being fully non-blocking under the hood.

KEY POINTS

- 'await' only works inside an async function (or a top-level module)
- Use try/catch for errors, instead of chaining .catch()
- An async function always returns a Promise, even if you return a plain value
- Reads like synchronous code — far easier to follow than long .then() chains

EXAMPLE 1

day3.js

```
async function fetchUserData(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    const user = await res.json();
    return user;
  } catch (err) {
    console.log('Failed to load user:', err.message);
    return null;
  }
}

const user = await fetchUserData(42);
```

REAL-WORLD EXAMPLE

Example 2 — A Multi-Step Checkout Flow

A checkout process must charge the card, update inventory, and send a confirmation email — strictly in order, each depending on the last succeeding. async/await keeps this readable, and one try/catch handles a failure at any step.

checkout-flow.js

```
async function completeCheckout(cart, card) {
  try {
    const payment = await chargeCard(card, cart.total);
    await updateInventory(cart.items);
    await sendConfirmationEmail(cart.userEmail, payment.id);
    return { success: true, paymentId: payment.id };
  } catch (err) {
    await refundIfCharged(card);  // rollback on any failure
    return { success: false, error: err.message };
  }
}
```

Why it matters: *Real payment flows need exactly this ordered, rollback-aware structure — async/await makes a multi-step transaction readable top-to-bottom.*

Promise.all, race, allSettled & any

DEFINITION

Static Promise methods for running multiple promises at once, each combining the results differently: all() needs every one to succeed, race() takes whichever settles first, allSettled() always waits for all of them, and any() resolves on the first success.

KEY POINTS

- Promise.all — needs every result; rejects immediately if one fails
- Promise.race — settles as soon as the fastest promise settles (win or lose)
- Promise.allSettled — waits for all, never short-circuits on a failure
- Promise.any — resolves on the first success, ignores earlier rejections

EXAMPLE 1

day3.js

```
const requests = [
  fetch('/api/weather'),
  fetch('/api/news'),
  fetch('/api/stocks'),
];

// ❌ one failure rejects the WHOLE batch
const all = await Promise.all(requests);

// ✅ every result comes back, success or failure
const results = await Promise.allSettled(requests);
// [{status:'fulfilled', ...}, {status:'rejected',
...}]
```

REAL-WORLD EXAMPLE

Example 2 — A Resilient Dashboard with a Timeout Guard

A dashboard pulls data from three independent microservices in parallel. allSettled() means one failing service doesn't blank the whole page, and race() against a timeout promise keeps a slow service from hanging the UI forever.

dashboard.js

```
function withTimeout(promise, ms) {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error('timeout')), ms));
  return Promise.race([promise, timeout]);  // whichever finishes first
}

const widgets = await Promise.allSettled([
  withTimeout(getSalesWidget(), 3000),
  withTimeout(getTrafficWidget(), 3000),
  withTimeout(getAlertsWidget(), 3000),
]);
widgets.forEach((r, i) => r.status === 'fulfilled' ? renderWidget(i, r.value) : renderWidgetError(i));
```

Why it matters: *Real dashboards can't let one slow or broken microservice take the whole page down — this pattern is standard in production monitoring UIs.*

Generators & Iterators

DEFINITION

A generator function (function*) can pause its own execution with 'yield' and resume later, producing a sequence of values on demand. Generators implement the iterator protocol, so they work directly with for...of.

KEY POINTS

- function* + yield — pauses and returns one value per .next() call
- Each .next() call resumes right where the last yield left off
- Naturally usable in for...of loops, spread syntax, and destructuring
- Great for lazy sequences, infinite series, and custom iteration logic

EXAMPLE 1

day3.js

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

const gen = idGenerator();
gen.next().value;  // 1
gen.next().value;  // 2
gen.next().value;  // 3 — paused right where it left off
```

REAL-WORLD EXAMPLE

Example 2 — Lazily Paging Through an API

An async generator fetches only the next page of results when the caller actually asks for one via for await...of — instead of loading every page upfront, which matters a lot for large, paginated datasets.

paginate.js

```
async function* fetchAllPages(url) {
  let nextUrl = url;
  while (nextUrl) {
    const res = await fetch(nextUrl);
    const page = await res.json();
    yield page.items;  // pause here until asked for more
    nextUrl = page.nextPageUrl;
  }
}

for (await (const items of fetchAllPages('/api/orders')) {
  renderOrders(items);  // renders page-by-page as they arrive
}
```

Why it matters: *This is exactly how infinite-scroll feeds and large export tools stream data — never holding more in memory than the user has actually seen.*

RECAP

Day 3 in one glance

- ✓ Event loop: sync code first, then microtasks, then macrotasks
- ✓ Promises: pending → fulfilled/rejected, chainable with then/catch/finally
- ✓ async/await: Promises that read top-to-bottom, errors via try/catch
- ✓ all/race/allSettled/any: four ways to combine concurrent promises
- ✓ Generators: function* + yield pause and resume, power lazy sequences

Practice: refactor a nested-callback chain into async/await, and write a generator that lazily produces the Fibonacci sequence.

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 4: Arrays & Iterables

Definitions + two real-world examples for every concept

day4.js

```
// Run with: node day4.js
topics = ['array methods', 'destructuring', 'Set/Map'];
```

DAY4-ARRAYS-AND-ITERABLES/DAY4.JS

Today's 5 concepts

01 Core Array Methods — map, filter & reduce

02 Advanced Array Methods — flat, flatMap, find & every/some

03 Destructuring — arrays, objects, nested & defaults

04 Set & Map

05 Iterables & the Iterator Protocol

Each concept: a clear definition, then two real-world code examples.

Core Array Methods — map, filter & reduce

DEFINITION

The three foundational array transformation methods: map() turns each element into something new, filter() keeps only elements that pass a test, and reduce() folds the whole array down into a single value.

KEY POINTS

- map() — always returns a new array of the SAME length
- filter() — returns a new array, possibly shorter
- reduce() — folds everything into one value: a sum, object, or array
- None of them mutate the original array — always return new data

EXAMPLE 1

day4.js

```
const prices = [10, 25, 8, 40];

const withTax = prices.map((p) => p * 1.1);
const affordable = prices.filter((p) => p < 30);
const total = prices.reduce((sum, p) => sum + p, 0);

withTax;      // [11, 27.5, 8.8, 44]
affordable;   // [10, 25, 8]
total;        // 83
```

REAL-WORLD EXAMPLE

Example 2 — Building a Shopping Cart Summary

A checkout page needs the cart's total price, a list of any out-of-stock items, and cleaned-up line items for display — three separate array operations chained together on the same raw cart data.

cart-summary.js

```
const cart = [
  { name: 'Headphones', price: 59, inStock: true },
  { name: 'Charger', price: 19, inStock: false },
];

const outOfStock = cart.filter((item) => !item.inStock);

const lineItems = cart
  .filter((item) => item.inStock)
  .map((item) => `${item.name} — $${item.price.toFixed(2)}`);

const subtotal = cart
  .filter((item) => item.inStock)
  .reduce((sum, item) => sum + item.price, 0);
```

Why it matters: *This filter → map → reduce chain is the everyday backbone of e-commerce carts, invoices, and any UI that summarizes a list of records.*

flat, flatMap, find & every/some

DEFINITION

Beyond map/filter/reduce, JS arrays have specialized helpers: flat() flattens nested arrays, flatMap() maps then flattens one level in a single pass, find()/findIndex() locate a single element, and some()/every() run quick boolean checks.

KEY POINTS

- flat(depth) — flattens nested arrays; default depth is 1
- flatMap() — a map() and a flat(1), combined efficiently
- find()/findIndex() — first match, or undefined/-1 if none
- some() — at least one passes; every() — all of them pass

EXAMPLE 1

day4.js

```
const nested = [[1,2], [3,4], [5]];
nested.flat();  // [1, 2, 3, 4, 5]

const sentences = ['hi there', 'bye now'];
sentences.flatMap((s) => s.split(' '));
// ['hi','there','bye','now']

const nums = [4, 9, 15, 20];
nums.find((n) => n > 10);   // 15
nums.every((n) => n > 0);  // true
nums.some((n) => n > 18);  // true
```

REAL-WORLD EXAMPLE

Example 2 — Validating a Signup Form's Fields

A signup form tracks a list of fields with their own validity. every() gates the submit button, some() shows a form-level error banner, and find() jumps focus to the very first invalid field.

form-validation.js

```
const fields = [
  { name: 'email', valid: true },
  { name: 'password', valid: false },
  { name: 'age', valid: true },
];

const canSubmit = fields.every((f) => f.valid);
const hasErrors = fields.some((f) => !f.valid);
const firstInvalid = fields.find((f) => !f.valid);

submitButton.disabled = !canSubmit;
if (firstInvalid) focusField(firstInvalid.name);  // jumps to 'password'
```

Why it matters: *This exact trio — every/some/find — is how real forms decide when to enable Submit, show a warning, and guide the user to the actual problem.*

Destructuring — Arrays, Objects & Defaults

DEFINITION

Destructuring unpacks values from arrays or properties from objects into distinct variables in one expression. It supports skipping items, renaming, default values, and reaching into nested structures.

KEY POINTS

- Array destructuring unpacks by position — skip items with extra commas
- Object destructuring unpacks by property name, can rename with ':'
- Default values apply only when the value is undefined
- Nested destructuring reaches directly into objects inside objects

EXAMPLE 1

day4.js

```
// array destructuring — skip + default
const [first, , third = 'N/A'] = ['a', 'b'];
// first = 'a', third = 'N/A'

// object destructuring — rename + nested + default
const {
  name: fullName,
  address: { city },
  role = 'guest',
} = { name: 'Asha', address: { city: 'Pune' } };
// fullName = 'Asha', city = 'Pune', role = 'guest'
```

REAL-WORLD EXAMPLE

Example 2 — Unpacking an API Response Cleanly

A user-profile component only needs a handful of fields from a much larger API payload. Destructuring pulls exactly what's needed, renames a field for clarity, and falls back to a default when a field is missing.

profile-card.js

```
const apiResponse = {
  id: 42,
  full_name: 'Ravi Shah',
  contact: { email: 'ravi@mail.com' },
};

function ProfileCard({
  full_name: name,
  contact: { email },
  avatarUrl = '/default-avatar.png',
}) {
  return `${name} · ${email} · ${avatarUrl}`;
}

ProfileCard(apiResponse);
```

Why it matters: *Destructuring straight in a function's parameters is the standard way React and Vue components pull exactly the props they need from a larger object.*

Set & Map

DEFINITION

Set stores unique values of any type — no duplicates allowed. Map stores key-value pairs where a key can be ANY type, not just a string like a plain object requires. Both keep insertion order and are directly iterable.

KEY POINTS

- Set — automatic deduplication; .has() is a fast O(1) lookup
- Map — .get()/.set()/.has()/.delete(); keys can be objects, functions, etc.
- Both work directly with for...of and spread syntax
- WeakSet/WeakMap — same idea, object-only keys, allow garbage collection

EXAMPLE 1

day4.js

```
// Set — dedupe a list instantly
const tags = ['js', 'css', 'js'];
const unique = [...new Set(tags)];  // ['js','css']

// Map — keys of any type
const userScores = new Map();
userScores.set('asha', 98);
userScores.set('ravi', 84);
userScores.get('asha');  // 98
userScores.size;    // 2
```

REAL-WORLD EXAMPLE

Example 2 — Caching API Calls & Deduplicating a Live Feed

A Map caches responses by request URL so the same endpoint is never fetched twice. A Set tracks event IDs already rendered in a live feed, so a duplicate WebSocket message doesn't render the same item twice.

cache-and-feed.js

```
// ✅ Map as a request cache
const cache = new Map();
async function getCached(url) {
  if (cache.has(url)) return cache.get(url);
  const data = await (await fetch(url)).json();
  cache.set(url, data);
  return data;
}

// ✅ Set to drop duplicate live-feed events
const seenIds = new Set();
socket.on('event', (e) => {
  if (seenIds.has(e.id)) return;  // skip duplicate
  seenIds.add(e.id);
  renderEvent(e);
});
```

Why it matters: *This is exactly how real apps avoid refetching the same data twice and keep live feeds (chat, notifications) from showing duplicate items.*

Iterables & the Iterator Protocol

DEFINITION

An iterable is any object implementing Symbol.iterator, making it work with for...of, spread syntax, and destructuring. Arrays, Strings, Maps, and Sets are built-in iterables — and you can define the protocol on your own objects too.

KEY POINTS

- for...of works on ANY iterable — arrays, strings, Maps, Sets, generators
- Spread (...) and Array.from() both consume the iterator protocol
- Array-like objects (NodeList, arguments) aren't auto-iterable everywhere
- Add a [Symbol.iterator] method to make your own object iterable

EXAMPLE 1

day4.js

```
const range = {
  from: 1, to: 4,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next: () => current <= last
        ? { value: current++, done: false }
        : { value: undefined, done: true },
    };
  },
};

[...range];  // [1, 2, 3, 4]
```

REAL-WORLD EXAMPLE

Example 2 — Turning a NodeList Into a Real Array

document.querySelectorAll() returns a NodeList — iterable with for...of, but missing array methods like map and filter. Array.from() (which also relies on the iterator protocol) converts it into a true array in one line.

dom-list.js

```
const nodeList = document.querySelectorAll('.product-card');

// ✗ NodeList has no .map()
// nodeList.map(el => el.textContent);  // TypeError

// ✓ convert via the iterator protocol
const cardTexts = Array.from(nodeList, (el) => el.textContent.trim());

// ✓ or spread syntax — same protocol, different syntax
const cardsArray = [...nodeList];
const visibleCards = cardsArray.filter((el) => el.offsetParent !== null);
```

Why it matters: *This exact 'NodeList has no map()' surprise is one of the most common real bugs in vanilla-JS DOM code — Array.from() is the standard fix.*

RECAP

Day 4 in one glance

- ✓ map/filter/reduce: transform, narrow, and fold — never mutate
- ✓ flat/flatMap/find/every/some: flatten and test arrays precisely
- ✓ Destructuring: unpack arrays and objects, with defaults and renaming
- ✓ Set/Map: unique values and any-type keys, both directly iterable
- ✓ Iterables: Symbol.iterator powers for...of, spread, and Array.from

Practice: dedupe an array of objects by id using a Set, and write a custom iterable that yields even numbers only.

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 5: Types & Error Handling

Definitions + two real-world examples for every concept

day5.js

```
// Run with: node day5.js
topics = ['coercion', 'try/catch', 'custom errors'];
```

Today's 5 concepts

01 Type Coercion & Equality — == vs === and truthy/falsy

02 typeof, instanceof & Type-Checking Patterns

03 try/catch/finally & Custom Error Classes

04 Error Handling in Async Code

05 Global Error Handlers & Defensive Patterns

Each concept: a clear definition, then two real-world code examples.

Type Coercion & Equality

DEFINITION

JavaScript automatically converts values between types in many operations. The == operator coerces both sides to a common type before comparing; === compares type AND value with no coercion. Every value is also either 'truthy' or 'falsy' in a boolean context.

KEY POINTS

- Only 6 falsy values: false, 0, '', null, undefined, NaN — rest are truthy
- == coerces types (1 == '1' is true); === never does (1 === '1' is false)
- Default to === almost everywhere — it's predictable, == often surprises
- Coercion edge cases exist ([] == false is true) — another reason to avoid ==

EXAMPLE 1

day5.js

```
1 == '1';         // true  — coerced to compare
1 === '1';        // false — different types

null == undefined;  // true
null === undefined; // false

// falsy values — memorize these six:
Boolean(false);     // false
Boolean(0);         // false
Boolean('');        // false
Boolean(null);      // false
Boolean(NaN);       // false
Boolean([]);        // true  ← surprising!
```

REAL-WORLD EXAMPLE

Example 2 — The 'Falsy Zero' Bug in a Quantity Field

A cart form checks whether a quantity was entered using a plain falsy check. It breaks the moment someone legitimately sets quantity to 0 — a genuinely common production bug caused by mixing up 'empty' with 'falsy'.

quantity-check.js

```
// ✗ 0 is falsy — this treats a valid '0' as 'missing'
function validateQuantity(qty) {
  if (!qty) {
    return 'Quantity is required';  // fires even for qty = 0
  }
  return null;
}

// ✓ explicitly check for missing, not merely falsy
function validateQuantityFixed(qty) {
  if (qty === undefined || qty === null || qty === '') {
    return 'Quantity is required';
  }
  return null;  // qty = 0 now passes correctly
}
```

Why it matters: *Confusing 'falsy' with 'missing' is one of the single most common real bugs — it silently breaks any field where 0 or an empty array is a valid value.*

typeof, instanceof & Type-Checking

DEFINITION

typeof returns a string naming a value's primitive type. instanceof checks whether an object's prototype chain includes a given constructor. Together with Array.isArray() and explicit null checks, they're how JS code figures out what it's working with at runtime.

KEY POINTS

- typeof null === 'object' — a famous, long-standing JS quirk
- typeof [] === 'object' too — use Array.isArray() to detect arrays
- instanceof walks the prototype chain — great for custom classes/errors
- Robust type-checks usually combine several of these checks together

EXAMPLE 1

day5.js

```
typeof 42;          // 'number'
typeof 'hi';        // 'string'
typeof undefined;   // 'undefined'
typeof null;        // 'object'  ← the famous quirk
typeof [];          // 'object'  ← arrays too

Array.isArray([]);  // true — the real array check

class ApiError extends Error {}
const err = new ApiError('failed');
err instanceof Error;      // true
err instanceof ApiError;   // true
```

REAL-WORLD EXAMPLE

Example 2 — Safely Narrowing an Unknown API Payload

A generic API client receives JSON of unknown shape and must safely check its type before using it — exactly how real client libraries guard against malformed or unexpected server responses before handing data to the rest of the app.

safe-parse.js

```
function parseApiResult(data) {
  if (data instanceof Error) {
    throw data;  // already an Error object — rethrow
  }
  if (Array.isArray(data)) {
    return data.map(parseApiResult);  // recurse into a list
  }
  if (data === null || typeof data !== 'object') {
    return data;  // primitive — nothing to unwrap
  }
  return { ...data, receivedAt: new Date() };
}
```

Why it matters: *Real API clients can't trust server data blindly — this layered typeof/instanceof/Array.isArray check is standard defensive parsing before data reaches the UI.*

try/catch/finally & Custom Error Classes

DEFINITION

try/catch/finally lets you run code that might throw, handle the failure, and always run cleanup afterward. Custom Error subclasses attach extra context (like a status code) and let you distinguish error types cleanly with instanceof.

KEY POINTS

- try runs code; catch handles anything thrown; finally always runs
- class MyError extends Error — custom name, extra fields, instanceof checks
- Branch inside catch using instanceof to handle different errors differently
- Never swallow an error silently — always log it or rethrow it

EXAMPLE 1

day5.js

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

try {
  throw new ValidationError('email', 'Invalid email');
} catch (err) {
  if (err instanceof ValidationError) {
    highlightField(err.field);
  }
} finally {
  hideLoadingSpinner();  // always runs
}
```

REAL-WORLD EXAMPLE

Example 2 — Distinct Errors Driving Real UI Branching

An API layer throws different Error subclasses for different failure types. The calling UI code uses instanceof to react correctly: redirect to login on auth failure, show a 404 page on missing data, or a generic error otherwise.

api-errors.js

```
class UnauthorizedError extends Error {}
class NotFoundError extends Error {}

async function loadOrderPage(id) {
  try {
    const order = await fetchOrder(id);  // throws typed errors
    renderOrder(order);
  } catch (err) {
    if (err instanceof UnauthorizedError) {
      redirectToLogin();
    } else if (err instanceof NotFoundError) {
      render404Page();
    } else {
      renderGenericError();
    }
  }
}
```

Why it matters: *Typed errors plus instanceof branching is exactly how production apps show a login redirect vs a 404 page vs a generic error, from one catch block.*

Error Handling in Async Code

DEFINITION

Errors in Promises don't behave like synchronous throws — a rejected Promise with nothing watching it becomes an 'unhandled rejection' instead of crashing loudly. try/catch around await, and .catch() on every promise, are both needed for solid async error handling.

KEY POINTS

- A rejected Promise with no .catch() becomes an 'unhandledRejection'
- try/catch around 'await' works exactly like synchronous try/catch
- An error thrown inside .then() propagates to the next .catch()
- Always handle every promise — both awaited AND fire-and-forget ones

EXAMPLE 1

day5.js

```
async function getUserSafely(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error('Fetch failed');
    return await res.json();
  } catch (err) {
    logError(err);
    return null;  // caller gets null, not a crash
  }
}

// ✗ fire-and-forget with NO .catch — silent unhandled rejection
getUserSafely('bad-id');  // safe here — try/catch inside handles it
```

REAL-WORLD EXAMPLE

Example 2 — Fire-and-Forget Analytics That Never Crash the App

A page logs an analytics event after load without awaiting it, so the UI isn't blocked. If that network call fails, it must NOT surface as an unhandled rejection or break the page — handled defensively with its own .catch().

analytics.js

```
function logPageView(page) {
  // ✗ un-awaited promise with no .catch — risky
  // fetch('/api/track', { method: 'POST', body: page });

  // ✓ defensive fire-and-forget — errors handled locally
  fetch('/api/track', { method: 'POST', body: page })
    .catch((err) => {
      console.warn('Analytics failed silently:', err);
      // intentionally NOT rethrown — analytics is non-critical
    });
}

logPageView('/checkout');  // runs, page keeps working either way
```

Why it matters: *Non-critical background calls (analytics, telemetry, prefetching) must fail quietly — this pattern keeps a flaky third-party endpoint from ever breaking the page.*

Global Error Handlers & Defensive Patterns

DEFINITION

Beyond local try/catch, real apps register global handlers as a last-resort safety net: window 'error'/'unhandledrejection' events in browsers, or process.on('uncaughtException'/'unhandledRejection') in Node — to log and gracefully recover from anything that slips past local handling.

KEY POINTS

- Global handlers are a SAFETY NET, not a substitute for local try/catch
- Browser: window.addEventListener('error' / 'unhandledrejection', ...)
- Node: process.on('uncaughtException' / 'unhandledRejection', ...)
- Pair with a fallback UI so users see something graceful, not a blank page

EXAMPLE 1

day5.js

```
// browser: catch anything that slipped past local handling
window.addEventListener('error', (event) => {
  reportToMonitoring(event.error);
});

window.addEventListener('unhandledrejection', (event) => {
  reportToMonitoring(event.reason);
  event.preventDefault();  // stop the noisy console warning
});

// Node.js equivalent
process.on('uncaughtException', (err) =>
reportToMonitoring(err));
```

REAL-WORLD EXAMPLE

Example 2 — An Error Boundary Backed by a Global Safety Net

A UI component catches its own rendering errors and shows a friendly fallback instead of a blank screen. Global window handlers catch anything outside that render cycle — like a broken click handler — so nothing ever fails completely silently.

error-boundary.js

```
class ErrorBoundary extends Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(err) { reportToMonitoring(err); }
  render() {
    return this.state.hasError
      ? <FriendlyErrorScreen />
      : this.props.children;
  }
}

// global net — catches errors OUTSIDE React's render cycle too
window.addEventListener('error', (e) => reportToMonitoring(e.error));
```

Why it matters: *Layered defense — component-level boundaries plus a global net — is exactly how production apps avoid ever showing a fully blank, broken page to a user.*

RECAP

Day 5 in one glance

- ✓ Coercion: default to ===, know the 6 falsy values, watch for '0'
- ✓ typeof/instanceof: combine checks — null and arrays need special care
- ✓ try/catch/finally: custom Error classes make instanceof branching clean
- ✓ Async errors: try/catch around await, .catch() on every fire-and-forget call
- ✓ Global handlers: a safety net for anything that slips past local handling

Practice: write a custom NetworkError class, and fix a form's validation bug caused by a falsy-zero check.

{ }

ADVANCED JAVASCRIPT · 6-DAY COURSE

Day 6: Functional Programming & Design Patterns

Definitions + two real-world examples for every concept

day6.js

```
// Run with: node day6.js
topics = ['pure functions', 'compose', 'patterns'];
```

Today's 5 concepts

01 Pure Functions & Immutability

02 Function Composition — compose & pipe

03 Module & Singleton Patterns

04 Observer / Pub-Sub Pattern

05 Factory & Decorator Patterns

Each concept: a clear definition, then two real-world code examples.

Pure Functions & Immutability

DEFINITION

A pure function always returns the same output for the same input and causes no side effects — no mutating its arguments, no touching outside state. Immutability means never changing data in place; you always produce a new copy instead.

KEY POINTS

- Same input → same output, every single time, no exceptions
- No side effects — no logging, no network calls, no mutation inside
- Immutability: use spread/map/slice instead of push/splice/assignment
- Pure + immutable code is predictable, easy to test, and safe in parallel

EXAMPLE 1

day6.js

```
// ✗ impure — mutates the input array
function addItem(cart, item) {
  cart.push(item);  // mutates caller's array!
  return cart;
}

// ✓ pure — returns a new array, original untouched
function addItemPure(cart, item) {
  return [...cart, item];
}

const original = ['book'];
const updated = addItemPure(original, 'pen');
original;  // ['book']        — unchanged
updated;   // ['book','pen'] — new array
```

REAL-WORLD EXAMPLE

Example 2 — Why React/Redux Reducers Must Stay Pure

A Redux-style reducer that mutates state directly breaks change detection — React compares object references, so a mutated (but reference-equal) state object looks 'unchanged' and the UI silently fails to re-render.

cart-reducer.js

```
// ✗ mutates state directly — React won't notice the change
function cartReducerBroken(state, action) {
  if (action.type === 'ADD_ITEM') {
    state.items.push(action.item);  // same reference!
    return state;
  }
  return state;
}

// ✓ pure reducer — always returns a brand-new object
function cartReducerFixed(state, action) {
  if (action.type === 'ADD_ITEM') {
    return { ...state, items: [...state.items, action.item] };
  }
  return state;
}
```

Why it matters: *This exact reference-equality check is how React, Redux, and most state libraries detect changes — an impure reducer is one of the most common real bugs in these apps.*

Function Composition — compose & pipe

DEFINITION

Composition combines small, single-purpose functions into one larger operation by feeding one function's output directly into the next one's input — typically via pipe() (left-to-right) or compose() (right-to-left, the traditional math order).

KEY POINTS

- pipe(f, g, h)(x) === h(g(f(x))) — reads naturally left to right
- compose(f, g, h)(x) === f(g(h(x))) — right to left, the math convention
- Each function stays small, focused, and independently testable
- Building complex logic from simple, reusable pieces is a core FP habit

EXAMPLE 1

day6.js

```
const pipe = (...fns) => (x) =>
  fns.reduce((acc, fn) => fn(acc), x);

const trim = (s) => s.trim();
const toLower = (s) => s.toLowerCase();
const capitalize = (s) => s[0].toUpperCase() +
s.slice(1);

const cleanName = pipe(trim, toLower, capitalize);

cleanName('  aSHA  ');  // 'Asha'
```

REAL-WORLD EXAMPLE

Example 2 — A Composed CSV Import Pipeline

A bulk CSV import needs each row parsed, validated, normalized, and enriched with defaults — in that exact order. Composing four small functions into one pipeline keeps each step testable on its own and reusable across other import jobs.

import-pipeline.js

```
const parseRow = (line) => line.split(',');
const validateRow = (fields) => {
  if (fields.length < 3) throw new Error('Bad row');
  return fields;
};
const normalizeFields = ([name, email, qty]) => ({
  name: name.trim(), email: email.toLowerCase(), qty: Number(qty),
});
const enrichWithDefaults = (row) => ({ status: 'pending', ...row });

const importRow = pipe(parseRow, validateRow, normalizeFields, enrichWithDefaults);

importRow('Asha,ASHA@MAIL.COM,3');
```

Why it matters: *Real ETL and import jobs are almost always a pipeline of small transforms — composing them keeps each step independently testable instead of one giant tangled function.*

Module & Singleton Patterns

DEFINITION

The Module pattern exposes only a small public API while keeping implementation details private (via closures or ES Modules). The Singleton pattern guarantees a class or object has exactly one shared instance across the whole app.

KEY POINTS

- Module pattern: private state + small public API — the shape of any ES module
- Singleton: guarantees ONE shared instance, however many times it's 'created'
- Common Singleton uses: one DB connection, one global config, one cache
- Overusing Singleton hides dependencies — use it deliberately, not by default

EXAMPLE 1

day6.js

```
// Module pattern — closure keeps 'count' private
const counterModule = (() => {
  let count = 0;
  return { increment: () => ++count, get: () => count };
})();

// Singleton — always the same instance
class AppConfig {
  static #instance;
  static getInstance() {
    if (!AppConfig.#instance) AppConfig.#instance = new
AppConfig();
    return AppConfig.#instance;
  }
}
AppConfig.getInstance() === AppConfig.getInstance();  // true
```

REAL-WORLD EXAMPLE

Example 2 — One Shared Database Connection Pool

A Node.js API server must reuse a single database connection pool across every request, instead of opening a new one each time — a textbook Singleton use case that directly affects performance and connection limits.

db-connection.js

```
// db.js — exported once, reused everywhere it's imported
let pool = null;

function getPool() {
  if (!pool) {
    pool = createConnectionPool({ max: 10 });  // expensive to create
    console.log('Pool created once');
  }
  return pool;
}

module.exports = { getPool };

// orders.js and users.js both call getPool()
// and get the SAME pool — no duplicate connections
```

Why it matters: *Real backend services rely on exactly this pattern — opening a new connection pool per request would exhaust the database's connection limit almost immediately.*

Observer / Pub-Sub Pattern

DEFINITION

The Observer pattern lets a subject keep a list of dependents (observers) and notify all of them automatically when its state changes. Pub-Sub is a related variant where publishers and subscribers communicate through a shared channel without referencing each other directly.

KEY POINTS

- Subject keeps a list of subscriber callbacks and calls them all on change
- Decouples the code that CAUSES an event from the code that REACTS to it
- This exact mechanism powers DOM events, RxJS, and most UI state libraries
- Pub-Sub adds a channel/topic in between — publishers never know subscribers

EXAMPLE 1

day6.js

```
class EventBus {
  listeners = {};
  on(event, callback) {
    (this.listeners[event] ??= []).push(callback);
  }
  emit(event, data) {
    (this.listeners[event] || []).forEach((fn) => fn(data));
  }
}

const bus = new EventBus();
bus.on('order:placed', (order) => console.log('Got:',
order));
bus.emit('order:placed', { id: 42 });
```

REAL-WORLD EXAMPLE

Example 2 — One Cart Update Notifying Three Independent Widgets

Adding an item to the cart must update the header's item-count badge, the cart drawer, and the order summary — three unrelated UI widgets that each subscribe once, without the cart logic knowing any of them exist.

cart-events.js

```
const cartBus = new EventBus();

// three independent subscribers — each written separately
cartBus.on('item:added', () => updateHeaderBadgeCount());
cartBus.on('item:added', (item) => renderInCartDrawer(item));
cartBus.on('item:added', (item) => recalculateOrderSummary(item));

// the cart itself only knows how to emit — nothing else
function addToCart(item) {
  cart.push(item);
  cartBus.emit('item:added', item);
}
```

Why it matters: *This decoupling is exactly why you can add a brand-new widget (like a 'recently added' toast) later without ever touching addToCart's own code.*

Factory & Decorator Patterns

DEFINITION

A Factory function creates and returns objects without exposing the exact class or constructor logic to the caller. A Decorator wraps an existing object or function to add new behavior without modifying its original code.

KEY POINTS

- Factory — one function decides WHICH type of object to build, based on input
- Decorator — wraps a function/object, adding behavior before/after it runs
- Both favor composition over editing existing classes directly
- Decorators are the same idea as Day 1's higher-order function wrappers

EXAMPLE 1

day6.js

```
// Factory — hides which class gets built
function createNotification(type, message) {
  if (type === 'email') return new
EmailNotification(message);
  if (type === 'sms') return new SmsNotification(message);
  return new PushNotification(message);
}

// Decorator — wraps a function, adds behavior around it
function withLogging(fn) {
  return (...args) => {
    console.log('calling', fn.name);
    return fn(...args);
  };
}
```

REAL-WORLD EXAMPLE

Example 2 — A Themed Button Factory Plus an Analytics Decorator

A design system's button factory builds differently-styled buttons from one call, and a separate decorator adds click-tracking to ANY button's handler — without either piece needing to know about the other.

design-system.js

```
// Factory — one entry point for every button style
function createButton(variant, label, onClick) {
  const styles = { primary: 'btn-primary', danger: 'btn-danger' };
  return { label, className: styles[variant] ?? 'btn-default', onClick };
}

// Decorator — adds tracking to ANY click handler
function withAnalytics(handler, eventName) {
  return (...args) => {
    trackEvent(eventName);
    return handler(...args);
  };
}

const checkoutBtn = createButton('primary', 'Checkout',
  withAnalytics(handleCheckout, 'checkout_clicked'));
```

Why it matters: *This factory-plus-decorator combo is exactly how real design systems produce consistent components while layering in tracking or logging without touching component code.*

RECAP

Day 6 in one glance

- ✓ Pure functions: same input → same output, never mutate arguments
- ✓ Composition: pipe/compose turn small functions into readable pipelines
- ✓ Module/Singleton: private state behind a public API, one shared instance
- ✓ Observer/Pub-Sub: decouple the event source from everything reacting to it
- ✓ Factory/Decorator: build objects flexibly, add behavior without editing code

Practice: refactor an impure cart function to be pure, and build a small pub-sub logger that any part of an app can subscribe to.

{ }

ADVANCED JAVASCRIPT · 8-DAY COURSE

Day 7: Functions in Depth & Performance Patterns

Definitions + two real-world examples for every concept

day7.js

```
// Run with: node day7.js
topics = ['this', 'currying', 'debounce', 'memoize'];
```

DAY 7 - FUNCTIONS-IN-DEPTH-AND-PERFORMANCE

Today's 5 concepts

01 'this' Binding — call, apply & bind

02 Currying & Partial Application

03 Debounce & Throttle

04 Memoization & Caching Results

05 Tagged Template Literals

Each concept: a clear definition, then two real-world code examples.

'this' Binding — call, apply & bind

DEFINITION

In JavaScript 'this' is decided by HOW a function is called, not where it was written. call() and apply() invoke a function with an explicit 'this'; bind() returns a new function permanently locked to one. Arrow functions have no 'this' of their own — they inherit it.

KEY POINTS

- Method call — 'this' is whatever sits before the dot
- Bare call — 'this' is undefined in strict mode, globalThis otherwise
- call(thisArg, ...args) / apply(thisArg, argsArray) / bind returns a copy
- Arrow functions capture 'this' lexically and can never be re-bound

EXAMPLE 1

day7.js

```
const user = {
  name: 'Asha',
  greet() { return `Hi, ${this.name}`; },
};

user.greet();          // 'Hi, Asha'

const loose = user.greet;
loose();               // ✗ 'Hi, undefined' — lost 'this'

const bound = user.greet.bind(user);
bound();               // ✓ 'Hi, Asha' — locked forever

user.greet.call({ name: 'Ravi' });    // 'Hi, Ravi'
user.greet.apply({ name: 'Meera' });  // 'Hi, Meera'
```

REAL-WORLD EXAMPLE

Example 2 — The Lost 'this' in an Event Handler

A class method passed straight to addEventListener is called by the DOM, not by your object — so 'this' is no longer the instance and every field read blows up. Binding, or an arrow class field, fixes it.

search-box.js

```
class SearchBox {
  constructor(input) { this.query = ''; this.input = input; }

  onType(event) { this.query = event.target.value; }   // needs 'this'

  mount() {
    // ✗ bare reference — the DOM calls it, so 'this' is the input element
    // this.input.addEventListener('input', this.onType);

    // ✓ bind returns a copy permanently attached to this instance
    this.input.addEventListener('input', this.onType.bind(this));
  }
}

// ✓ modern alternative — an arrow class field can never lose 'this'
class SearchBoxModern {
  query = '';
  onType = (event) => { this.query = event.target.value; };
}
```

Why it matters: *"Cannot read properties of undefined" inside a click or input handler is almost always a lost 'this' — bind() or an arrow class field is the standard fix.*

Currying & Partial Application

DEFINITION

Currying rewrites f(a, b, c) as f(a)(b)(c) — one argument at a time, returning a new function until every argument has arrived. Partial application locks in some arguments now and leaves the rest for later.

KEY POINTS

- Curried functions return functions until the arity is satisfied
- fn.bind(null, a) is partial application built into the language
- Turns one general function into many pre-configured specialised ones
- Fits perfectly into the pipe/compose pipelines from Day 6

EXAMPLE 1

day7.js

```
const curry = (fn) => (...args) =>
  args.length >= fn.length
    ? fn(...args)
    : curry(fn.bind(null, ...args));

const add = (a, b, c) => a + b + c;
const cAdd = curry(add);

cAdd(1)(2)(3);      // 6
cAdd(1, 2)(3);      // 6  — partial then finish

// partial application with bind
const add10 = add.bind(null, 10);
add10(5, 1);        // 16
```

REAL-WORLD EXAMPLE

Example 2 — An API Client Configured Once, Used Everywhere

Every request to the same service shares a base URL, headers and auth. Currying bakes those in once, so callers only supply what actually changes — the path and the body.

api-client.js

```
const request = (baseUrl) => (token) => (method) => (path, body) =>
  fetch(`${baseUrl}${path}`, {
    method,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: body && JSON.stringify(body),
  });

const api  = request('https://api.shop.com/v2')(session.token);
const get  = api('GET');
const post = api('POST');

get('/orders');                      // no headers repeated
post('/orders', { itemId: 42 });     // no base URL repeated
```

Why it matters: *This is how real HTTP clients, loggers and feature-flag helpers get configured once at startup and then reused across hundreds of call sites without repeating options.*

Debounce & Throttle

DEFINITION

Two closures that limit how often an expensive function runs. Debounce waits until the activity STOPS for N milliseconds and then runs once. Throttle allows at most one run per N-millisecond window while the activity keeps going.

KEY POINTS

- Debounce — run after silence: search boxes, autosave, resize end
- Throttle — run at a steady rate: scroll, mousemove, drag
- Both are just a closure over a timer id — Day 1 closures in production
- Clear the timer on unmount, or a stale call fires into a dead component

EXAMPLE 1

day7.js

```
function debounce(fn, wait = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);            // cancel the previous plan
    timer = setTimeout(() => fn(...args), wait);
  };
}

function throttle(fn, limit = 200) {
  let ready = true;
  return (...args) => {
    if (!ready) return;             // inside the cooldown
    ready = false;
    fn(...args);
    setTimeout(() => (ready = true), limit);
  };
}
```

REAL-WORLD EXAMPLE

Example 2 — A Type-Ahead Search That Doesn't Flood the API

An unguarded input handler fires one request per keystroke — twelve requests to type 'headphones'. Debounce collapses that into a single call, while throttle keeps a scroll handler from running on every frame.

search-and-scroll.js

```
const searchInput = document.querySelector('#search');

// ✗ one network request per keystroke — 12 requests for 'headphones'
// searchInput.addEventListener('input', (e) => searchApi(e.target.value));

// ✓ exactly one request, 300ms after the user stops typing
searchInput.addEventListener(
  'input',
  debounce((e) => searchApi(e.target.value), 300),
);

// ✓ scroll fires ~60x per second; this caps the work at 5x per second
window.addEventListener('scroll', throttle(() => maybeLoadNextPage(), 200));
```

Why it matters: *An un-debounced search box multiplies your API bill and rate-limits real users; an un-throttled scroll handler drops frames on mid-range phones.*

Memoization & Caching Results

DEFINITION

Memoization stores a function's result against its arguments, so a repeat call with the same input returns instantly instead of recomputing. It is only safe for pure functions — the same input must always deserve the same output.

KEY POINTS

- A Map keyed by the serialised arguments is the usual cache
- Only memoize PURE functions (Day 6) — cached side effects are bugs
- Huge wins for recursion, parsing, formatting and layout maths
- Bound the cache with a size limit or TTL, or it becomes a memory leak

EXAMPLE 1

day7.js

```
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);   // hit
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));

fib(40);   // instant — every sub-result is cached
// the same function without memoize takes seconds
```

REAL-WORLD EXAMPLE

Example 2 — Deduplicating Concurrent Requests by Caching the Promise

Three widgets mount at the same moment and all ask for user 42. Caching the resolved value is too late — the trick is to cache the pending Promise itself, so every caller waits on one shared request.

fetch-once.js

```
const inFlight = new Map();

function fetchUserOnce(id) {
  // ✓ cache the PROMISE, not the value — later callers join the same request
  if (inFlight.has(id)) return inFlight.get(id);

  const pending = fetch(`/api/users/${id}`)
    .then((res) => res.json())
    .finally(() => setTimeout(() => inFlight.delete(id), 60000));  // TTL

  inFlight.set(id, pending);
  return pending;
}

// header, sidebar and profile card all call this in the same tick
await Promise.all([fetchUserOnce(42), fetchUserOnce(42), fetchUserOnce(42)]);
// → exactly ONE network request
```

Why it matters: *Caching the promise rather than the result is what stops three components mounting together from firing three identical requests — the pattern behind React Query, SWR and Apollo.*

Tagged Template Literals

DEFINITION

Putting a function name directly in front of a template literal calls that function with the static string pieces in one array and the interpolated values in another — so you can inspect, escape or transform every value before it reaches the output.

KEY POINTS

- tag(strings, ...values) — strings.raw holds the un-escaped text
- Values arrive separately from the text, so each one can be sanitised
- Powers styled-components, graphql``, sql`` and i18n libraries
- The right tool wherever plain interpolation would be unsafe

EXAMPLE 1

day7.js

```
function highlight(strings, ...values) {
  return strings.reduce(
    (out, str, i) =>
      out + str + (i < values.length ? `**${values[i]}**` : ''),
    '',
  );
}

const name = 'Asha';
const count = 3;

highlight`Hi ${name}, you have ${count} new orders`;
// 'Hi **Asha**, you have **3** new orders'
```

REAL-WORLD EXAMPLE

Example 2 — A Tag That Escapes Every Interpolated Value

User-supplied text dropped straight into innerHTML is a script-injection hole. Because the tag receives the values separately from the markup, it can escape each one and still let you write natural HTML.

safe-html.js

```
const escapeHtml = (value) =>
  String(value)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;');

function safe(strings, ...values) {
  return strings.reduce(
    (out, str, i) => out + str + (i < values.length ? escapeHtml(values[i]) : ''),
    '',
  );
}

const comment = '<img src=x onerror=alert(1)>';

// ✗ raw interpolation — the injected tag really executes
// el.innerHTML = `<p>${comment}</p>`;

// ✓ the tag escapes every value before it touches the DOM
el.innerHTML = safe`<p>${comment}</p>`;
```

Why it matters: *Seeing the values apart from the static text is exactly why template-tag libraries can guarantee escaping that plain string concatenation never can.*

RECAP

Day 7 in one glance

- ✓ 'this': decided by HOW a function is called — bind or use arrows
- ✓ Currying: pre-fill arguments to build specialised functions
- ✓ Debounce waits for silence; throttle caps the rate
- ✓ Memoize pure functions — and cache promises to dedupe requests
- ✓ Tagged templates: see the values separately, so you can escape them

Practice: *wrap a search input with your own debounce(), and write a memoize() that expires entries after 60 seconds.*

{ }

ADVANCED JAVASCRIPT · 8-DAY COURSE

Day 8: Modern JavaScript Power Features

Definitions + two real-world examples for every concept

day8.js

```
// Run with: node day8.js
topics = ['?.', 'Symbol', 'Proxy', 'modules', 'abort'];
```

DAY 8 - MODERN-JAVASCRIPT-POWER-FEATURES

Today's 5 concepts

01 Optional Chaining, ?? & Logical Assignment

02 Symbols & Well-Known Symbols

03 Proxy & Reflect — Metaprogramming

04 ES Modules, Dynamic import() & Tree Shaking

05 AbortController — Cancelling Async Work

Each concept: a clear definition, then two real-world code examples.

Optional Chaining, ?? & Logical Assignment

DEFINITION

?. reads through a chain that may be missing and short-circuits to undefined instead of throwing. ?? supplies a fallback only for null or undefined — unlike ||, which also replaces 0, '' and false. ??=, ||= and &&= assign conditionally.

KEY POINTS

- a?.b?.c stops at the first null/undefined — no TypeError
- Works for calls and indexes too: obj.fn?.() and arr?.[i]
- 0 ?? 10 is 0, but 0 || 10 is 10 — that difference causes real bugs
- opts.retries ??= 3 sets a default only when nothing was provided

EXAMPLE 1

day8.js

```
const res = { data: { user: { address: null } } };

res.data.user.address.city;      // ✗ TypeError
res.data?.user?.address?.city;   // ✓ undefined

const settings = { volume: 0 };
settings.volume || 10;           // ✗ 10 — 0 is a valid volume!
settings.volume ?? 10;           // ✓ 0

const opts = {};
opts.retries ??= 3;              // only if null/undefined
opts.retries;                    // 3

res.onDone?.();                  // called only if it exists
```

REAL-WORLD EXAMPLE

Example 2 — Rendering a Deeply Nested API Payload Safely

A profile component reads five levels into a response that may be partial, missing or an error shape. Optional chaining removes the guard pyramid, and ?? keeps legitimate zeros and empty strings intact.

profile.js

```
// ✗ the old guard pyramid — verbose and easy to get wrong
// const city = res && res.data && res.data.user && res.data.user.address
//   && res.data.user.address.city || 'Unknown';

function renderProfile(res) {
  const city   = res?.data?.user?.address?.city ?? 'Unknown';
  const avatar = res?.data?.user?.avatarUrl ?? '/default-avatar.png';
  const posts  = res?.data?.user?.posts?.length ?? 0;   // ✓ 0 survives

  res?.onRendered?.();     // optional callback, invoked only if passed

  return `${city} · ${posts} posts · ${avatar}`;
}

renderProfile({});          // 'Unknown · 0 posts · /default-avatar.png'
renderProfile(undefined);   // same — still no crash
```

Why it matters: *This trio replaces the res && res.data && … pyramid and, crucially, stops a || default from silently overwriting a real 0 or empty string.*

Symbols & Well-Known Symbols

DEFINITION

A Symbol is a unique, immutable primitive used as a property key that can never collide with any other key. Well-known symbols such as Symbol.iterator and Symbol.toPrimitive let your own objects hook directly into built-in language behaviour.

KEY POINTS

- Symbol('id') !== Symbol('id') — every symbol is unique forever
- Symbol keys are skipped by Object.keys, for...in and JSON.stringify
- Symbol.for('key') looks up a shared, cross-file global registry
- Well-known: iterator, asyncIterator, toPrimitive, toStringTag, hasInstance

EXAMPLE 1

day8.js

```
const ID = Symbol('id');
const user = { name: 'Asha', [ID]: 9021 };

Object.keys(user);        // ['name']  — symbol key hidden
JSON.stringify(user);     // '{"name":"Asha"}'
user[ID];                 // 9021 — still reachable

// a well-known symbol changes how the object converts
const price = {
  amount: 250,
  [Symbol.toPrimitive](hint) {
    return hint === 'string' ? `$${this.amount}` : this.amount;
  },
};

`${price}`;               // '$250'
price + 50;               // 300
```

REAL-WORLD EXAMPLE

Example 2 — Library Metadata That Never Collides With User Data

A caching layer needs to tag the objects it has already seen. A normal string key could clash with a real field and would leak into every JSON payload; a symbol key stays completely invisible to application code.

cache-tag.js

```
// Symbol.for uses the global registry, so every file gets the SAME symbol
const CACHE_META = Symbol.for('app.cache.meta');

function tagEntry(record, meta) {
  record[CACHE_META] = { ...meta, cachedAt: Date.now() };
  return record;
}

const order = tagEntry({ id: 42, total: 999 }, { source: 'api' });

Object.keys(order);            // ['id','total'] — app code never sees the tag
JSON.stringify(order);         // '{"id":42,"total":999}' — not serialised
order[CACHE_META].source;      // 'api' — but the library can still read it

// ✗ a plain string key would show up in every payload and every for...in loop
// record.__cacheMeta = { ... };
```

Why it matters: *Libraries need bookkeeping on your objects without breaking your JSON responses or your key loops — symbol keys are the standard, collision-proof way to do it.*

Proxy & Reflect — Metaprogramming

DEFINITION

A Proxy wraps a target object and intercepts fundamental operations — get, set, has, deleteProperty — through traps you supply. Reflect holds the matching default behaviours, so a trap can add its own logic and then forward the operation unchanged.

KEY POINTS

- new Proxy(target, handler) — each handler method is called a 'trap'
- Reflect.get/set/has mirror the traps and perform the default action
- Powers Vue 3 reactivity, validation layers, mocks and audit logging
- Traps run on every single access — keep them cheap

EXAMPLE 1

day8.js

```
const target = { name: 'Asha', role: 'admin' };

const guarded = new Proxy(target, {
  get(obj, key) {
    if (!(key in obj)) {
      throw new ReferenceError(`No field: ${String(key)}`);
    }
    return Reflect.get(obj, key);   // default behaviour
  },
  set(obj, key, value) {
    if (key === 'role' && !['admin', 'editor'].includes(value)) {
      throw new TypeError('Invalid role');
    }
    return Reflect.set(obj, key, value);
  },
});

guarded.role = 'editor';   // ✓ allowed
guarded.typo;              // ✗ ReferenceError, not silent undefined
```

REAL-WORLD EXAMPLE

Example 2 — Reactive State, the Way Vue 3 Does It

Instead of asking developers to call setState, a framework can wrap the state object in a Proxy: any assignment is intercepted, applied through Reflect, and then used to trigger a re-render automatically.

reactive.js

```
function reactive(state, onChange) {
  return new Proxy(state, {
    set(obj, key, value) {
      const ok = Reflect.set(obj, key, value);   // do the real assignment
      onChange(key, value);                      // then react to it
      return ok;
    },
    deleteProperty(obj, key) {
      const ok = Reflect.deleteProperty(obj, key);
      onChange(key, undefined);
      return ok;
    },
  });
}

const cart = reactive({ items: [], total: 0 }, (key) => rerender(key));

cart.total = 999;      // ✓ rerender('total') fires — plain assignment, no setState
delete cart.coupon;    // ✓ rerender('coupon') fires too
```

Why it matters: *This trap-plus-Reflect pattern is literally the core of Vue 3's reactivity system, and of most validation, mocking and audit-logging wrappers.*

ES Modules, import() & Tree Shaking

DEFINITION

ES Modules are the standard way to split JavaScript across files: export bindings from one file and import them in another. Static imports are hoisted and analysable, which lets bundlers delete unused exports; import() loads a module lazily and returns a Promise.

KEY POINTS

- Named exports plus at most one default export per module
- Modules are singletons — evaluated once, cached, always strict mode
- import() returns a Promise, enabling code splitting and lazy routes
- Prefer named exports: they tree-shake far more reliably than defaults

EXAMPLE 1

day8.js

```
// math.js
export const add = (a, b) => a + b;
export const mul = (a, b) => a * b;
export default function sum(list) { return list.reduce(add, 0); }

// app.js
import sum, { add } from './math.js';

add(2, 3);      // 5
sum([1, 2, 3]); // 6
// 'mul' is never imported → the bundler drops it from the build

// lazy — loaded only when this line actually runs
const { default: Chart } = await import('./heavy-chart.js');
```

REAL-WORLD EXAMPLE

Example 2 — Lazy-Loading a Heavy Editor Only When It's Needed

A rich-text editor is 400 KB but only a small fraction of visitors ever write a review. Moving it behind import() keeps it out of the initial bundle and fetches it on the click that needs it.

lazy-editor.js

```
// ✗ 400 KB shipped to every visitor, parsed before the page is interactive
// import { Editor } from './rich-text-editor.js';

// ✓ fetched only when someone actually clicks 'Write a review'
reviewButton.addEventListener('click', async () => {
  showSpinner();
  try {
    const { Editor } = await import('./rich-text-editor.js');
    new Editor(container).mount();
  } catch (err) {
    showErrorBanner('Editor failed to load');   // Day 5: async errors
  } finally {
    hideSpinner();                              // Day 3: always cleans up
  }
});
```

Why it matters: *Moving rarely used features behind import() is the single biggest lever on first-load time — and since it is just a Promise, every Day 3 and Day 5 rule still applies to it.*

AbortController — Cancelling Async Work

DEFINITION

An AbortController produces a signal you can hand to fetch, event listeners and your own async helpers. Calling controller.abort() rejects the pending operation with an AbortError, so you can cancel stale work instead of racing it.

KEY POINTS

- const c = new AbortController(); fetch(url, { signal: c.signal })
- Aborting rejects with an AbortError — check err.name and ignore it
- AbortSignal.timeout(ms) gives a signal that cancels itself
- Cancel on unmount, or a late response repaints a screen the user has left

EXAMPLE 1

day8.js

```
const controller = new AbortController();

fetch('/api/search?q=head', { signal: controller.signal })
  .then((res) => res.json())
  .catch((err) => {
    if (err.name === 'AbortError') return;   // ✓ expected — ignore
    showErrorBanner(err.message);
  });

controller.abort();      // cancels the request in flight

// a signal that cancels itself after 3 seconds
fetch('/api/slow', { signal: AbortSignal.timeout(3000) });

// listeners can be removed the same way
el.addEventListener('scroll', onScroll, { signal: controller.signal });
```

REAL-WORLD EXAMPLE

Example 2 — Making Sure the Newest Search Result Always Wins

Without cancellation, a slow response for 'he' can arrive after the fast one for 'headphones' and overwrite it. Aborting the previous request on every keystroke guarantees only the latest term can render.

type-ahead.js

```
let controller;

async function search(term) {
  controller?.abort();                    // ✓ cancel the previous keystroke
  controller = new AbortController();

  try {
    const res = await fetch(`/api/search?q=${term}`, {
      signal: controller.signal,
    });
    renderResults(await res.json());      // only the newest term reaches here
  } catch (err) {
    if (err.name !== 'AbortError') renderError(err);   // ignore cancellations
  }
}

// debounce (Day 7) + abort = the complete production type-ahead
searchInput.addEventListener('input', debounce((e) => search(e.target.value), 300));
```

Why it matters: *Debounce alone reduces the number of requests; only cancellation stops an old, slow response from landing last and painting the wrong results.*

RECAP

Day 8 in one glance

- ✓ ?. and ?? : safe reads and defaults that respect 0, '' and false
- ✓ Symbols: collision-proof keys that JSON and Object.keys ignore
- ✓ Proxy + Reflect: intercept get/set — the core of Vue 3 reactivity
- ✓ Modules: static imports tree-shake, import() splits the bundle
- ✓ AbortController: cancel stale work so old responses can't win

Practice: *rewrite a nested && chain with ?. and ??, and add AbortController cancellation to a type-ahead search.*